Timing analysis on FPGA

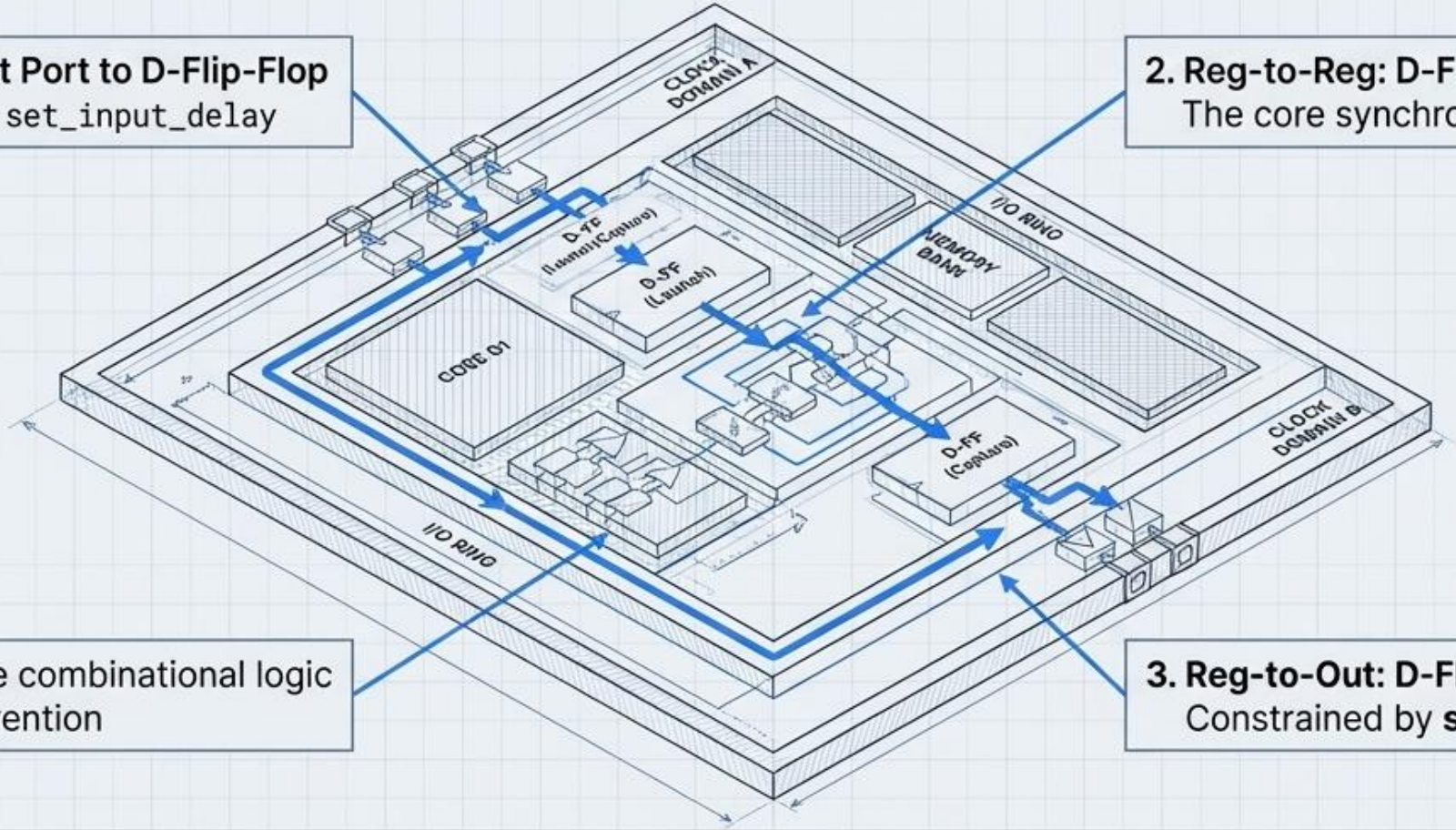
Why timing matters

- Even if the logic function is correct, the circuit may still fail if data arrives too late or changes too early.
- Functional correctness and timing correctness are different.

The Four Atomic Data Paths

1. In-to-Reg: Input Port to D-Flip-Flop
Constrained by `set_input_delay`

2. Reg-to-Reg: D-Flip-Flop to D-Flip-Flop
The core synchronous timing path



4. In-to-Out: Pure combinational logic
No clock intervention

3. Reg-to-Out: D-Flip-Flop to Output Port
Constrained by **set_output_delay**

The Core Definition: A timing path strictly originates at a launch pin (clock) and terminates at a capture pin (data, enable, or reset).

NotebookLM

In a synchronous FPGA design…

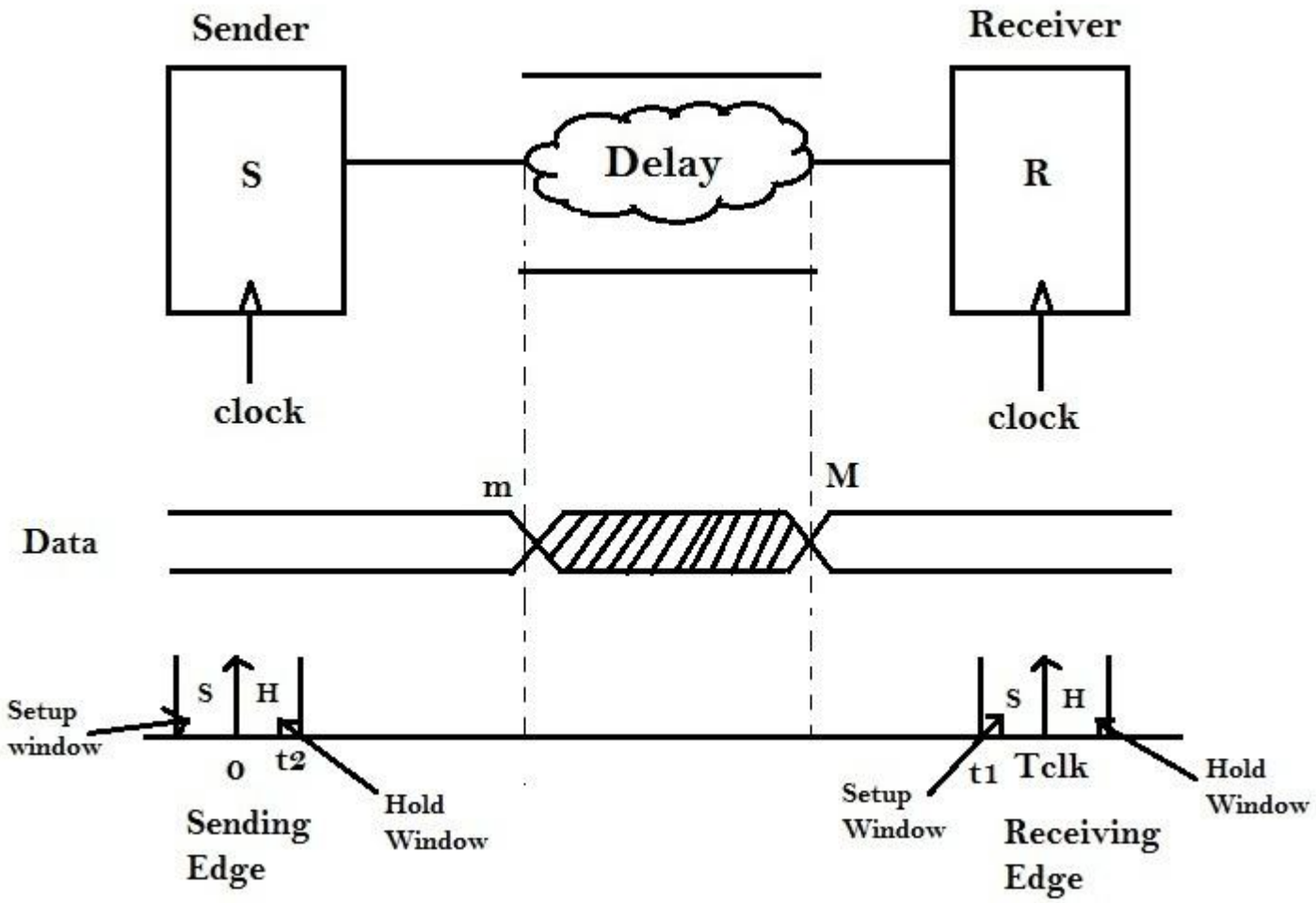
data moves from one flip-flop to another on clock edges. STA checks whether the data can travel in time.”

- FPGA has **flip-flops**
- FPGA has **combinational logic** like LUTs, adders, multipliers
- FPGA has **interconnect/routing**
- data travels through these blocks

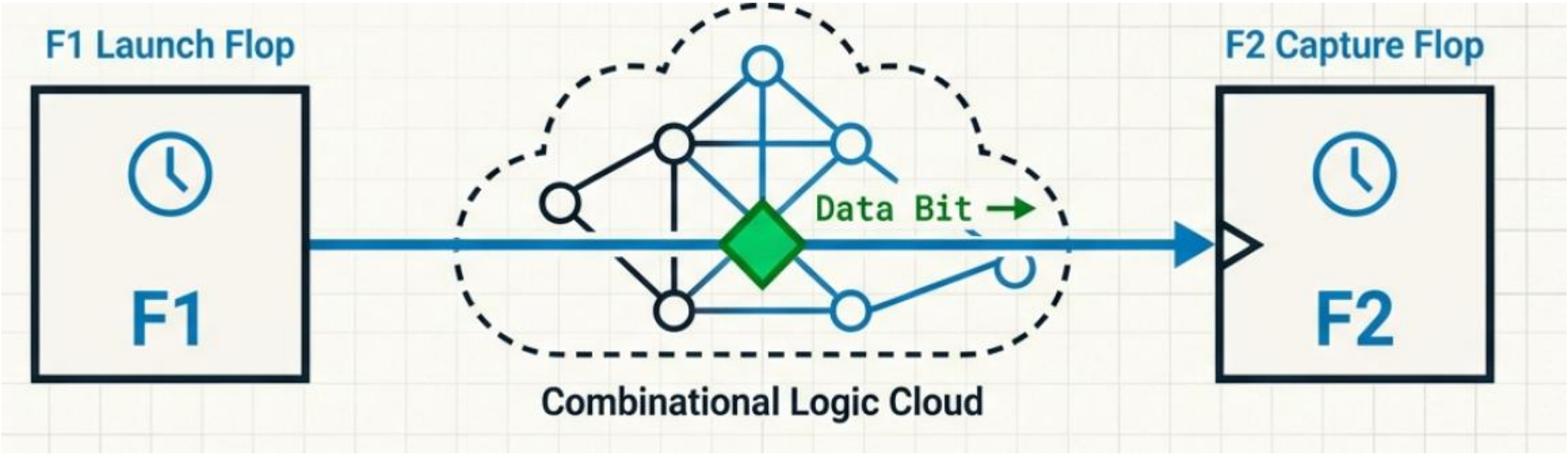
So the basic path is:

```
Register → Combinational Logic → Register
```

This is called a timing path.



Launch and Capture concept



- At one clock edge, **FF1 launches** data
- Data passes through logic and routing
- At the next clock edge, **FF2 captures** data

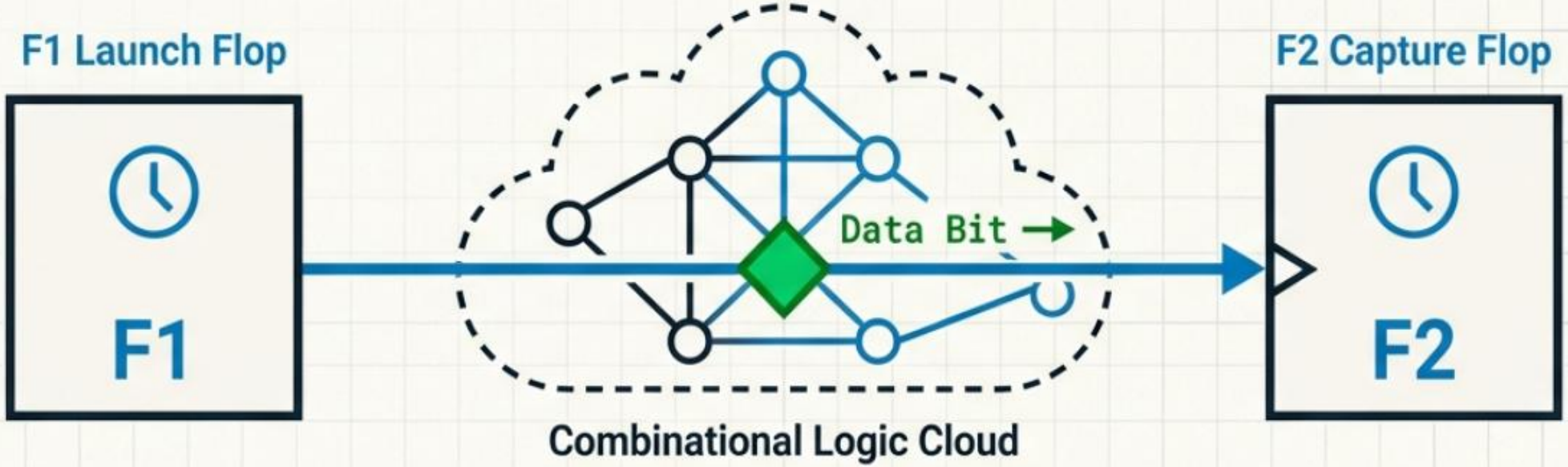
- **Launch edge**
- **Capture edge**

This is the foundation of STA.

Static Timing Analysis, or STA

- Static Timing Analysis, or STA, is used to **verify whether data in a synchronous FPGA design can move from one flip-flop to another within the available clock period.**
- **In an FPGA, a typical path is register to combinational logic to register. At one clock edge, the source register launches data. The data then passes through LUT logic and routing, and at the next clock edge, the destination register captures it.**

The Core Objective of STA



The Mission

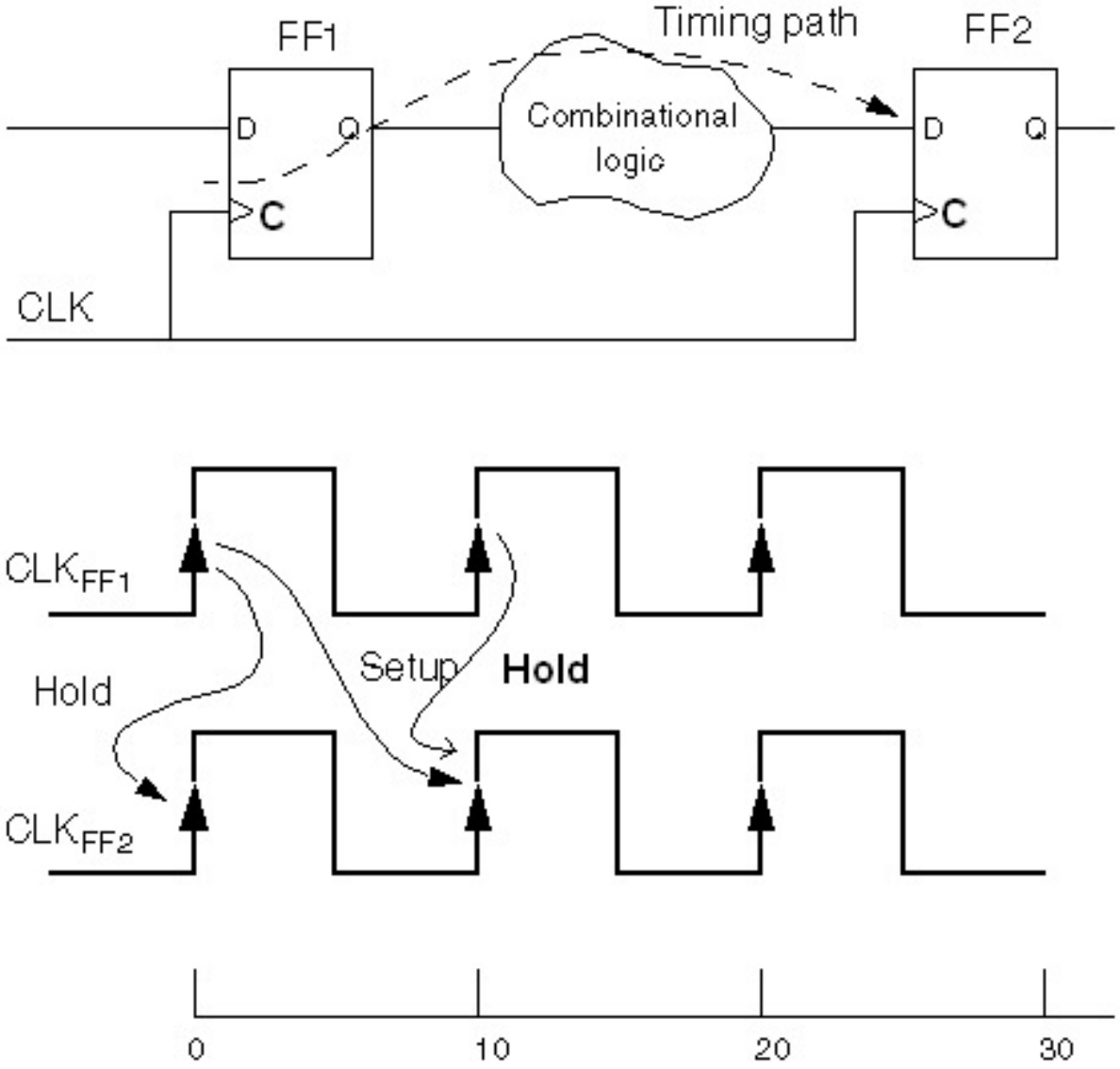
Ensure data launched by F1 successfully navigates physical delays to be safely captured by F2.

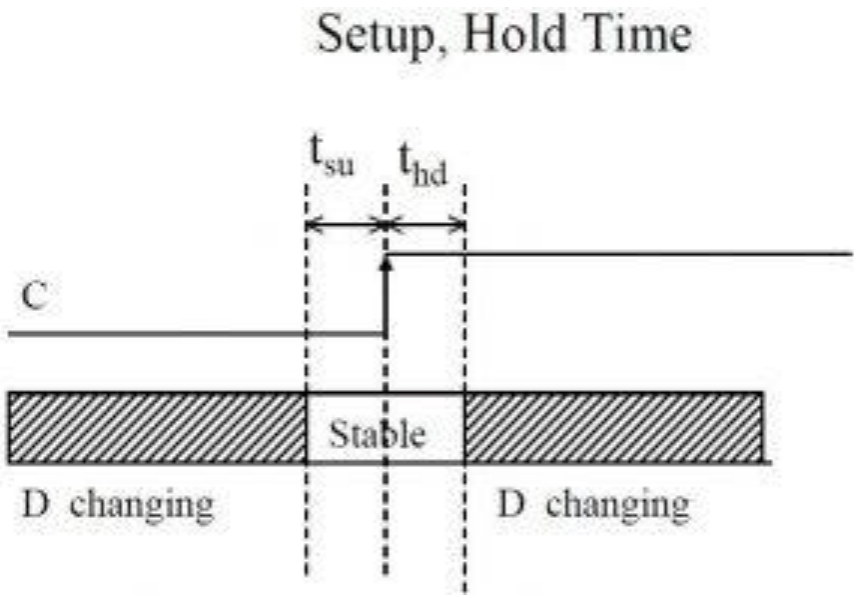
The Threat

If data arrives too late or moves too fast, the structural timing boundaries are breached.

The Consequence

The output enters a strictly undefined, metastable state, corrupting chip operation.

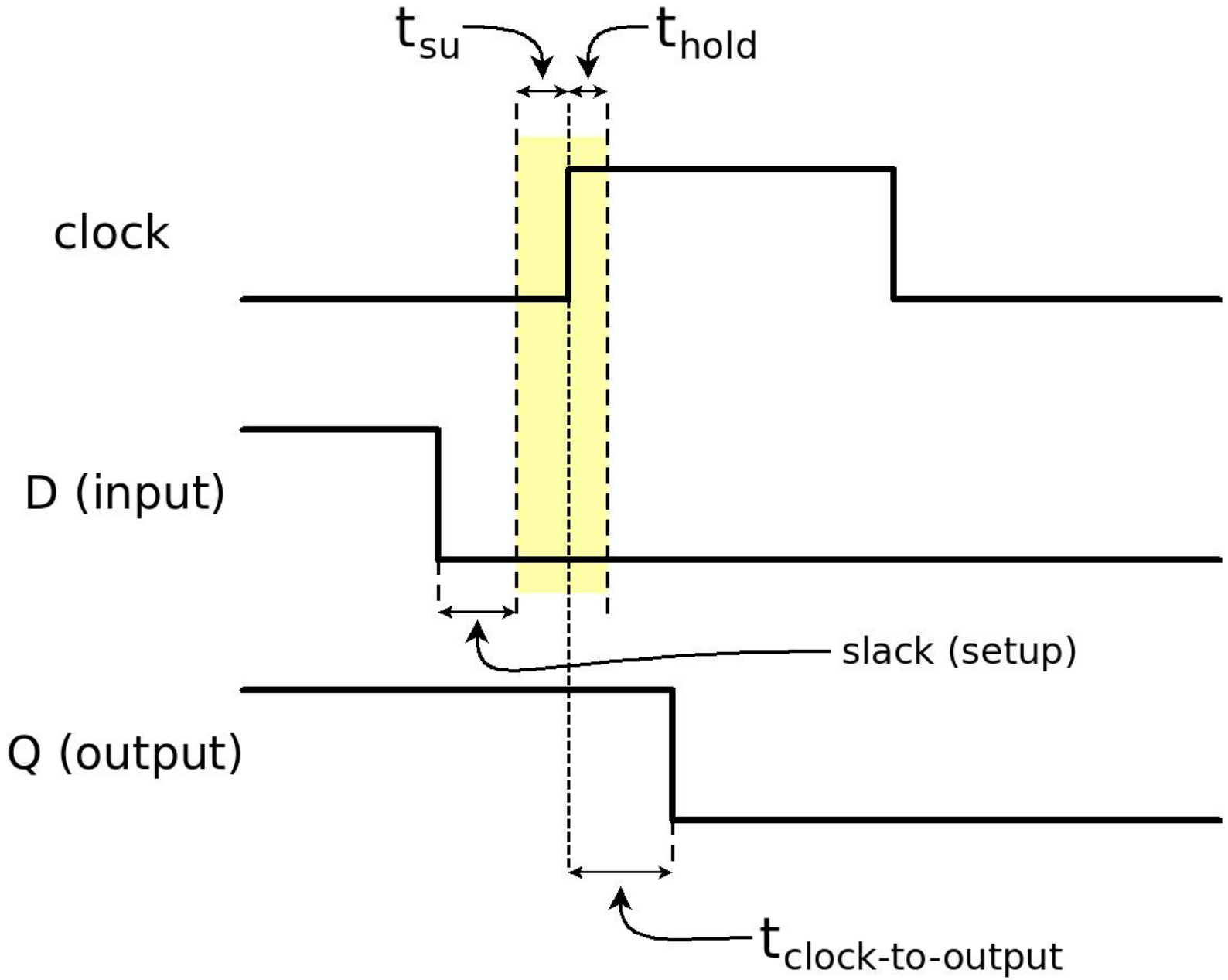

FF1
Timing path
FF2
D
Q
Combinational logic
D
Q
C
C
CLK
CLK$_{FF1}$
Hold
Setup
Hold
CLK$_{FF2}$
0
10
20
30


Setup, Hold Time
t_{su}
t_{hd}
C
Stable
D changing
D changing

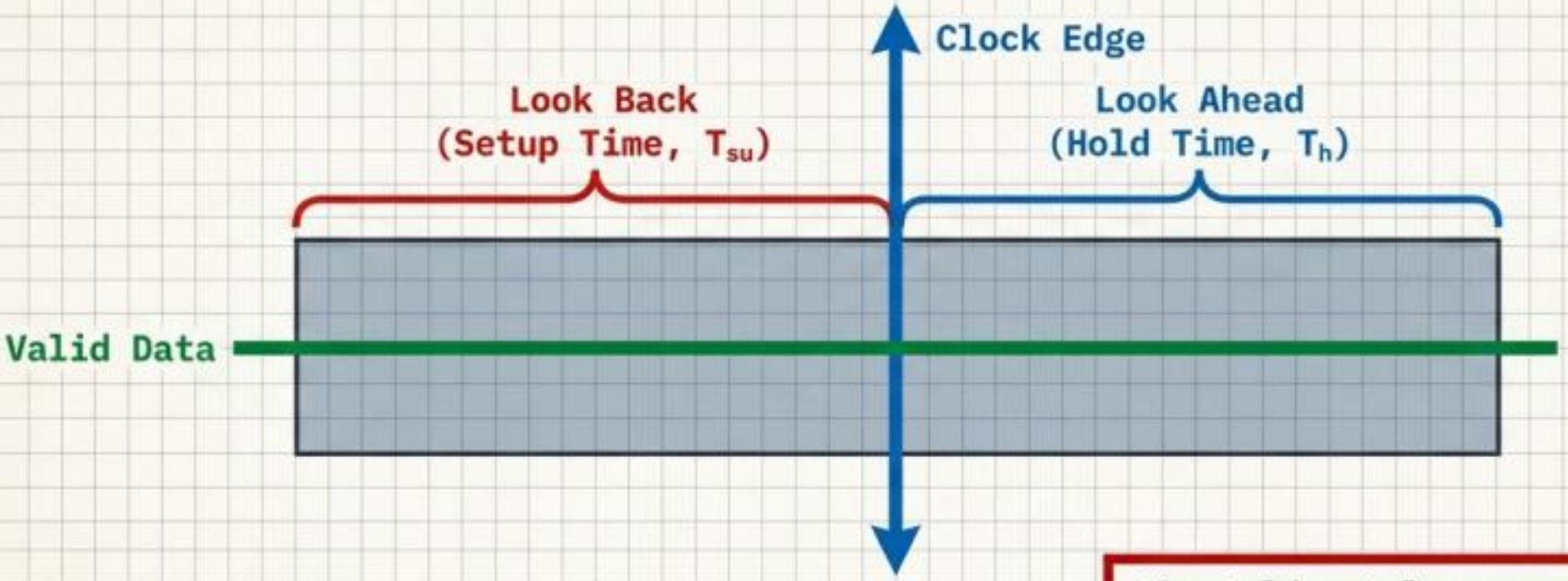
Setup and hold

In order to ensure that a flip-flop operates correctly and reliably, its inputs must be stable (i.e. not change its value) according to two timing requirements:

- The input must be stable during a period of time *before* the clock edge. This amount of time is called the *setup time*, and is marked t_{su}.
- The input must be stable during a period of time *after* the clock's edge. This amount of time is called the *hold time*, and is marked t_{hold}.


t_{su}
t_{hold}
clock
D (input)
slack (setup)
Q (output)
$t_{clock\text{-}to\text{-}output}$

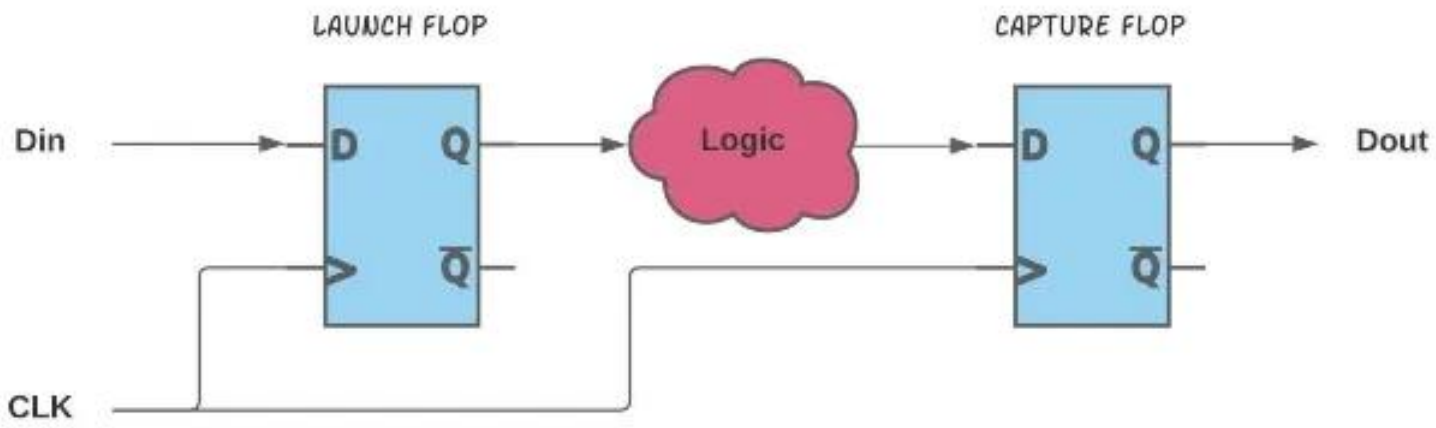
The Flip-Flop's Window of Stability

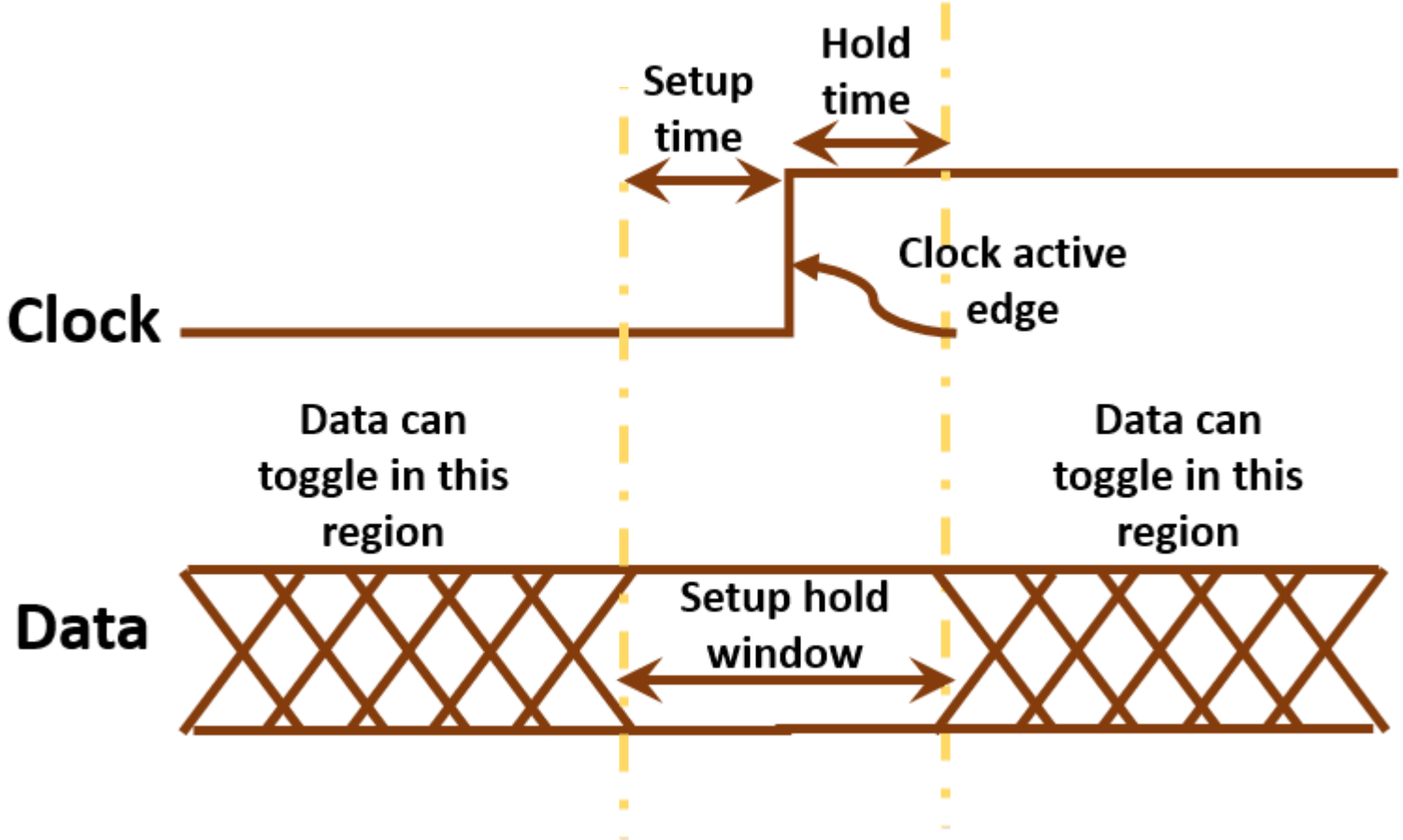


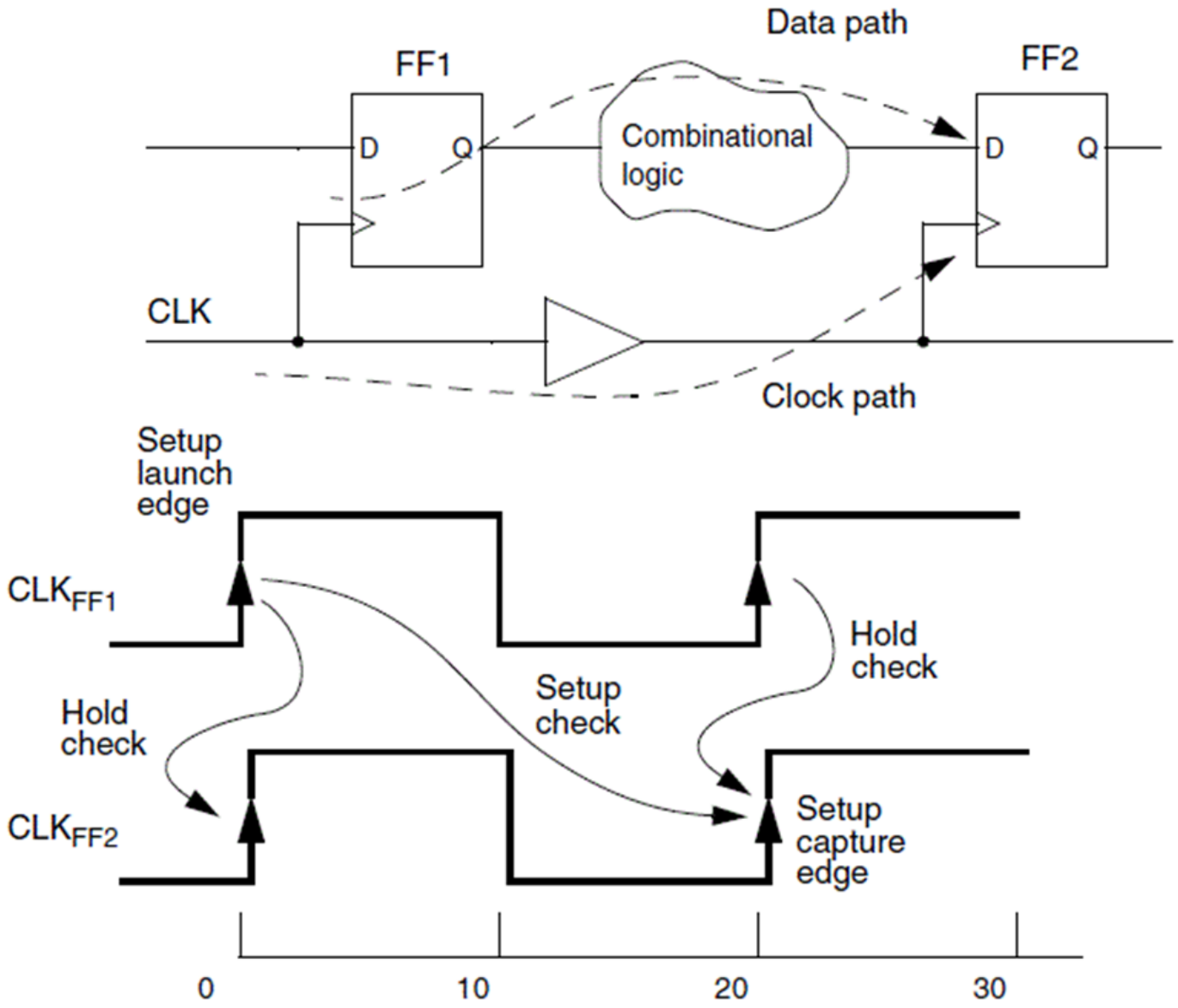
Look Back:
Data must be perfectly stable before the clock trigger strikes.

Look Ahead:
Data must remain stable after the clock trigger strikes.

The Golden Rule:
Any data transition (0->1 or 1->0) inside this shaded window results in an immediate timing violation. The bit is lost.


LAUNCH FLOP
CAPTURE FLOP
Din
D
Q
Q̄
Logic
D
Q
Q̄
Dout
CLK


Setup time
Hold time
Clock active edge
Clock
Data can toggle in this region
Data can toggle in this region
Data
Setup hold window


Data path
FF1
FF2
D
Q
Combinational logic
D
Q
CLK
Clock path
Setup launch edge
CLK$_{FF1}$
Hold check
Setup check
Hold check
CLK$_{FF2}$
Setup capture edge
0
10
20
30

Timing is measured between these two edges

Setup check

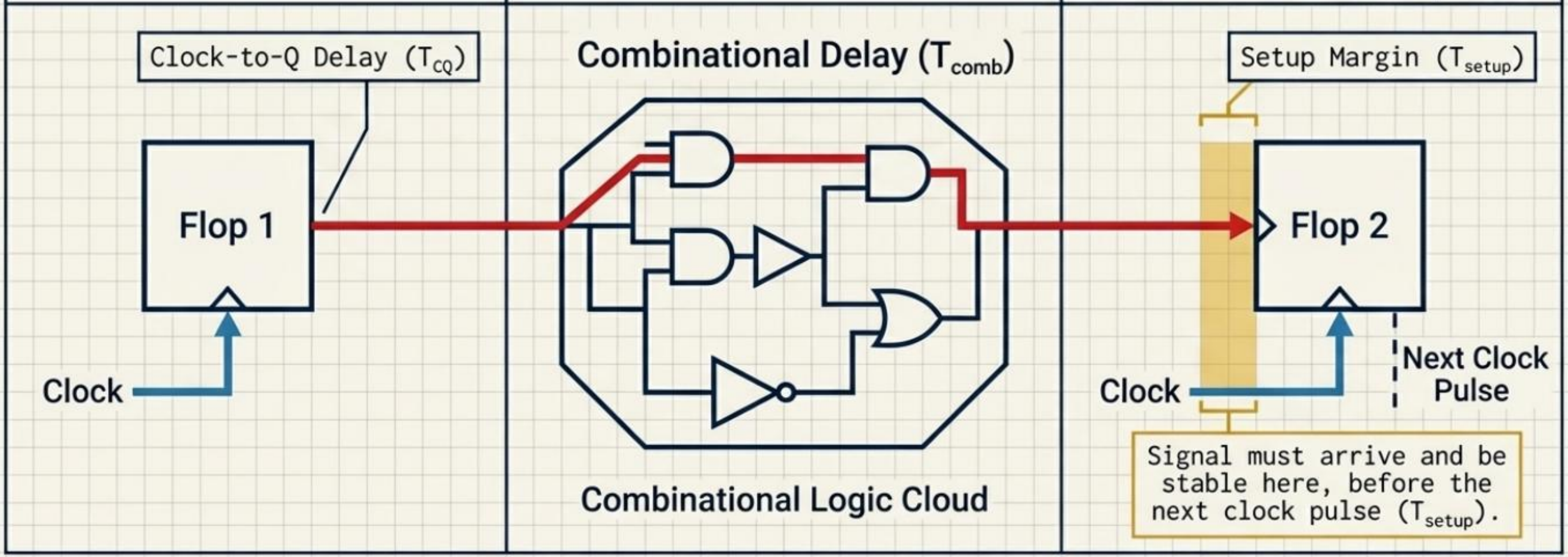
Hold check

The Anatomy of a Flop-to-Flop Path

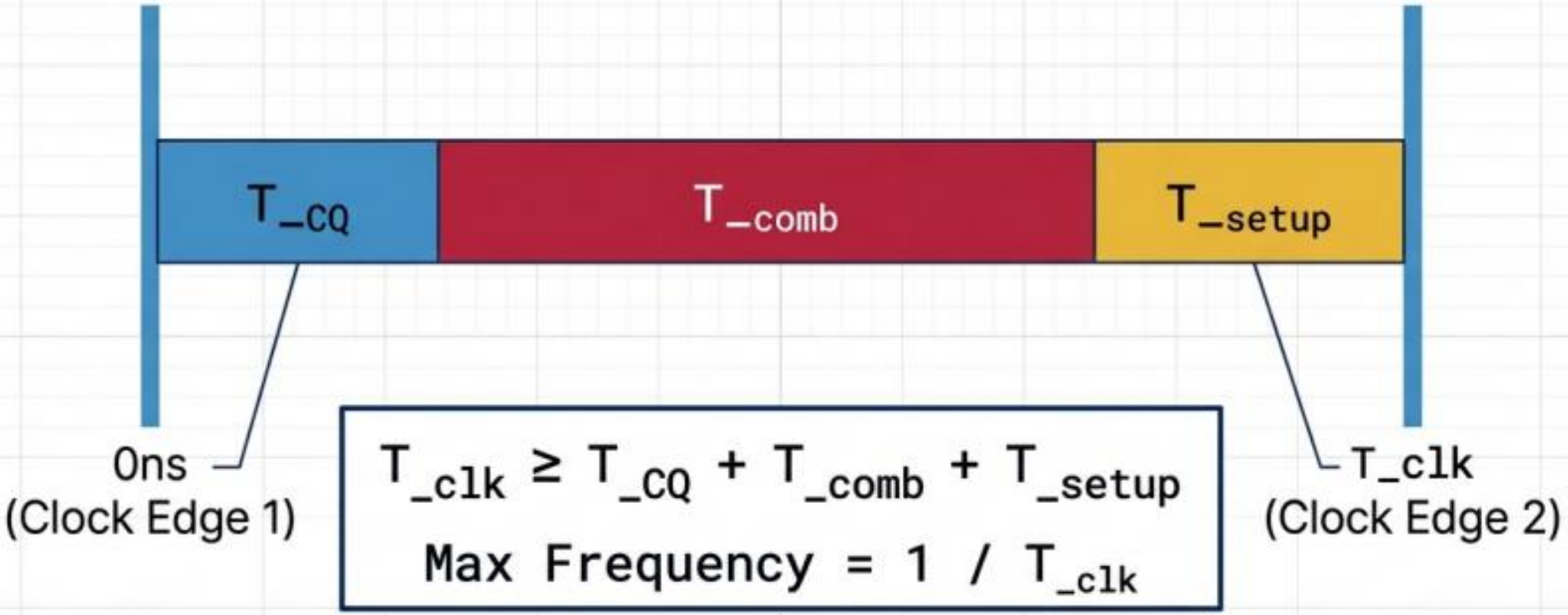
1. The signal requires T_{CQ} time just to physically exit the source flop after the clock hits.

2. The signal navigates the physical wires and logic gates, taking T_{comb} time.

3. The signal must arrive early enough to satisfy the destination's radar (T_{setup}) before the next clock pulse.



Deriving the Maximum Frequency (T_clk)



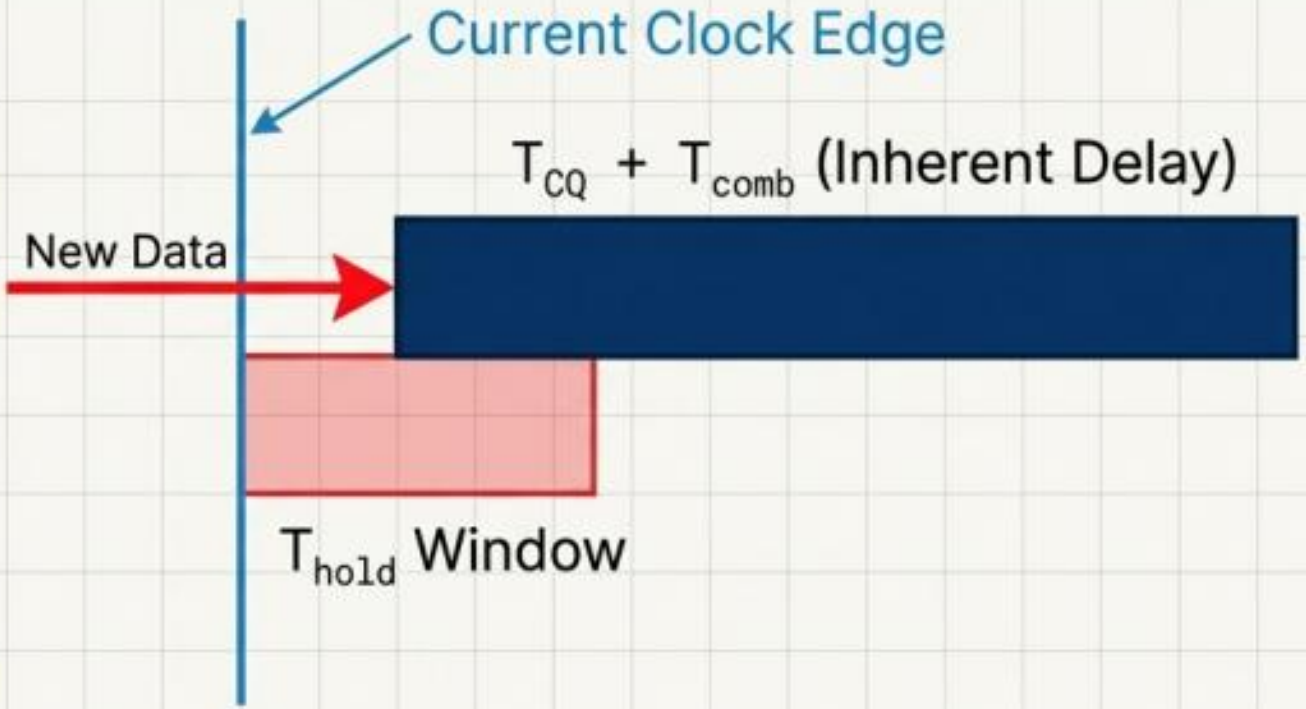
To avoid a Setup Violation, data must travel out of the source flop, through the logic, and set up at the destination BEFORE the next clock cycle arrives.

The Hold Constraint (Data Racing)

The Hold Margin Formula

$$T_{hold} \leq T_{CQ} + T_{comb}$$

To avoid a Hold Violation, data from the CURRENT cycle must not race through the logic so fast that it overwrites the previous data.
The physical delay of the circuit acts as a protective shield.



“In FPGA, combinational logic is implemented using LUTs, and delays also come from routing between resources.”

So in FPGA, delay mainly comes from:

- flip-flops
- LUTs
- routing
- sometimes DSP blocks, BRAM, etc.

“In many FPGA designs, routing delay is also a major part of total delay.”

Setup Time Constraint

$$T_{clk} \geq T_{cq} + T_{logic} + T_{route} + T_{setup}$$

Hold Time Constraint

$$T_{cq(min)} + T_{logic(min)} + T_{route(min)} \geq T_{hold}$$

- hold violation happens on **very fast paths**
- setup violation happens on **very slow paths**

Simple RTL example

```verilog
reg [7:0] a_reg, b_reg;
reg [8:0] sum;

always @(posedge clk) begin
    a_reg <= a;
    b_reg <= b;
end

always @(*) begin
    sum = a_reg + b_reg;
end

always @(posedge clk) begin
    y <= sum;
end
```

- `a_reg` and `b_reg` are launch registers
- `y` is capture register
- adder is the combinational path

How to check whether the FPGA board's clock will be supported by my design?

```tcl
create_clock -period 10 -name gclk [get_ports clk]
```

"This tells Vivado that one clock cycle is 10 ns, so the design must complete each valid register-to-register transfer within 10 ns."

- 10 ns means 100 MHz
- STA compares path delay against this clock period

Now Vivado STA tool will calculate… Slack!

Slack

- **Slack quantifies the timing margin of a path by measuring how early or late a data signal arrives at the capture register relative to the required time defined by clock constraints and setup/hold requirements.**

$$\text{Slack} = \text{Required Time} - \text{Arrival Time}$$

$$\text{Slack} = \text{Required Time} - \text{Arrival Time}$$

- **Required time** = when data is needed
- **Arrival time** = when data actually arrives

Interpretation:

- slack > 0 → timing met
- slack = 0 → exactly on limit
- slack < 0 → timing violation

Arrival Time (AT)

The time at which the data signal actually reaches the destination (capture register input).

$$AT = T_{cq} + T_{comb}$$

Required Time (RT)

The latest time by which the data must arrive to satisfy timing constraints.

$$RT = T_{clk} - T_{setup}$$

$$\boxed{\text{Slack}_{setup} = (T_{clk} - T_{setup}) - (T_{cq} + T_{comb})}$$

$$\boxed{\text{Slack}_{setup} = (T_{clk} - T_{setup}) - (T_{cq} + T_{comb})}$$

Slack value	Meaning
Slack > 0	Timing satisfied ✅
Slack = 0	Critical boundary ⚠️
Slack < 0	Timing violation ❌

Multiple RTL paths

```verilog
y1 <= a_reg + b_reg;
y2 <= a_reg * b_reg;
```

- register → adder → register
- register → multiplier → register

"STA automatically checks all timing paths. We do not manually analyze only one path."

Critical path

"In a design there are many timing paths. The slowest one is called the critical path."

```
Tclk(min) = maximum path delay
```

"Even if many paths are fast, the slowest path decides the maximum clock frequency."

What STA means by “static”

“STA does not apply input test vectors like simulation. It mathematically checks timing of all possible paths.”

- simulation checks behavior for chosen inputs
- STA checks timing for all paths without running waveforms

That is why it is called **Static Timing Analysis**.

Summary

- **For correct operation, the data must arrive before the capture edge by at least the setup time. This gives the setup condition: clock-to-Q delay plus logic delay plus routing delay plus setup time must be less than or equal to the clock period.**
- **Also, data should not change too quickly after the clock edge, otherwise, a hold violation can occur.**
- **STA is called static because it checks all timing paths mathematically without applying input test vectors. In Vivado, we define the clock using create_clock, and the tool automatically checks all paths and reports the worst one, called the critical path.**
- **The key quantity is slack, which is the required time minus the arrival time. Positive slack means timing is met, while negative slack means violation. Therefore, STA helps us determine whether the FPGA design can reliably run at the target clock frequency.**

Case study -1:

```verilog
reg [7:0] a_reg, b_reg;
reg [8:0] sum;

always @(posedge clk) begin
    a_reg <= a;
    b_reg <= b;
end

always @(*) begin
    sum = a_reg + b_reg;
end

always @(posedge clk) begin
    y <= sum;
end
```

Timing	**Setup**
Worst Negative Slack (WNS):	3.051 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	16

Timing	Setup
Worst Hold Slack (WHS):	0.306 ns
Total Hold Slack (THS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	16

Case-2

verilog

```verilog
module add_mult_two_path (
    input  wire        clk,
    input  wire        rst,
    input  wire [7:0]  a,
    input  wire [7:0]  b,
    output reg  [8:0]  y_add,
    output reg  [15:0] y_mult
);

    reg [7:0] a_reg;
    reg [7:0] b_reg;

    reg [8:0]  add_result;
    reg [15:0] mult_result;
```

```
// Input registers
always @(posedge clk or posedge rst) begin
    if (rst) begin
        a_reg <= 8'd0;
        b_reg <= 8'd0;
    end
    else begin
        a_reg <= a;
        b_reg <= b;
    end
end
```

```
// Combinational datapaths
always @(*) begin
    add_result  = a_reg + b_reg;
    mult_result = a_reg * b_reg;
end

// Output registers
always @(posedge clk or posedge rst) begin
    if (rst) begin
        y_add  <= 9'd0;
        y_mult <= 16'd0;
    end
    else begin
        y_add  <= add_result;
        y_mult <= mult_result;
    end
end
```

Path 1: Add path

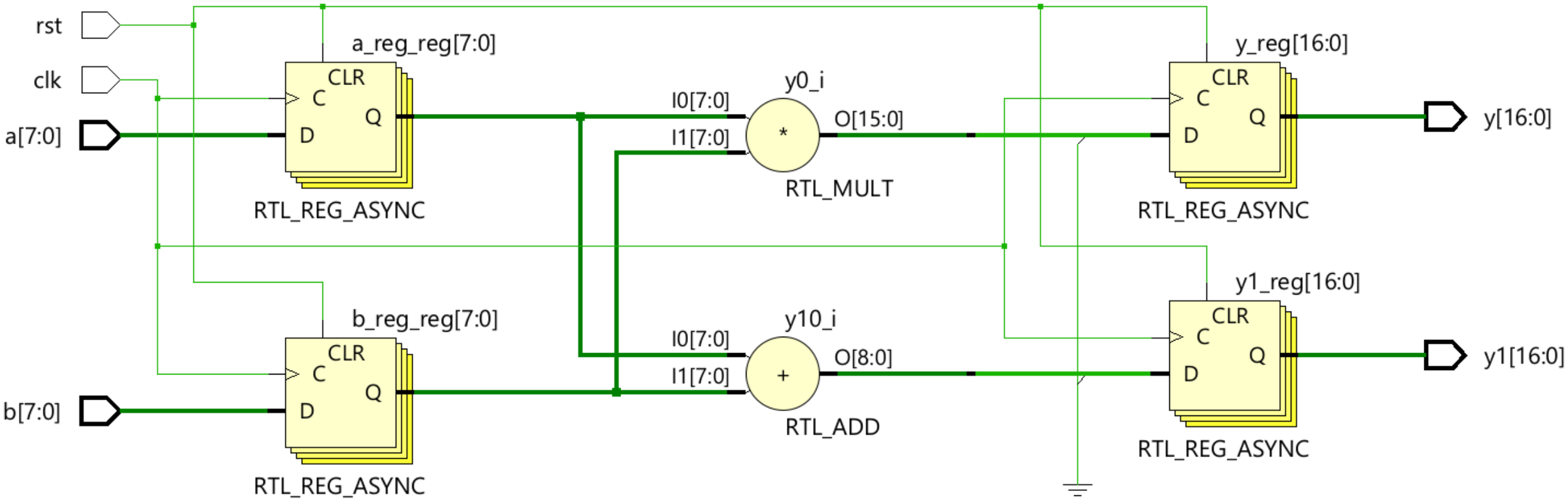
```
a_reg, b_reg → adder → y_add
```

Path 2: Multiply path

```
a_reg, b_reg → multiplier → y_mult
```

Vivado will analyze both paths automatically.

> “This module has two register-to-register datapaths. One goes through an adder and another goes through a multiplier. STA checks both paths, and the path with larger delay determines the maximum clock frequency.”


rst
clk
a[7:0]
b[7:0]
a_reg_reg[7:0]
CLR
C
D
Q
RTL_REG_ASYNC
b_reg_reg[7:0]
CLR
C
D
Q
RTL_REG_ASYNC
y0_i
I0[7:0]
I1[7:0]
*
O[15:0]
RTL_MULT
y10_i
I0[7:0]
I1[7:0]
+
O[8:0]
RTL_ADD
y_reg[16:0]
CLR
C
D
Q
RTL_REG_ASYNC
y1_reg[16:0]
CLR
C
D
Q
RTL_REG_ASYNC
y[16:0]
y1[16:0]

Timing

Setup | H

Worst Negative Slack (WNS):	2.926 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	25

Timing

Worst Hold Slack (WHS):	0.254 ns
Total Hold Slack (THS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	25

- **Design a Verilog module to implement $y = (ab + c)$ using only four registers: `a_reg`, `b_reg`, `c_reg`, and `y`. Synthesize and implement the design in Vivado.**
- **Apply a clock constraint and obtain the values of WNS and WHS from the timing report. Identify the critical path.**
- **Then modify the design using pipelining to improve timing slack. Re-run implementation and compare the WNS and WHS of the pipelined and non-pipelined versions.**

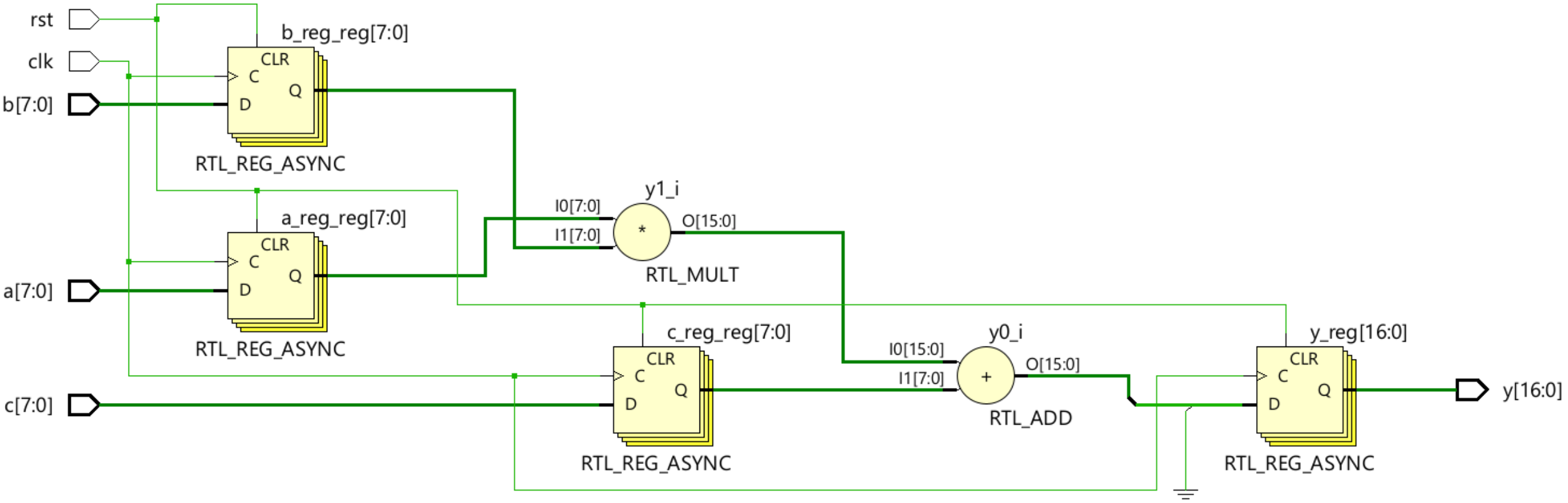
```
module mac (
   input  wire       clk,
   input  wire       rst,
   input  wire [7:0]  a,
   input  wire [7:0]  b,
   input  wire [7:0]  c,
   output reg  [16:0] y
);

   reg [7:0] a_reg, b_reg, c_reg;

   always @(posedge clk or posedge rst) begin
      if (rst) begin
         a_reg <= 8'd0;
         b_reg <= 8'd0;
         c_reg <= 8'd0;
         y     <= 17'd0;
      end
      else begin
         a_reg <= a;
         b_reg <= b;
         c_reg <= c;

         // Non-pipelined MAC
         y <= (a_reg * b_reg) + c_reg;
      end
   end

endmodule
```


rst
clk
b[7:0]
a[7:0]
c[7:0]
b_reg_reg[7:0]
CLR
C
D
Q
RTL_REG_ASYNC
a_reg_reg[7:0]
CLR
C
D
Q
RTL_REG_ASYNC
y1_i
I0[7:0]
I1[7:0]
*
O[15:0]
RTL_MULT
c_reg_reg[7:0]
CLR
C
D
Q
RTL_REG_ASYNC
y0_i
I0[15:0]
I1[7:0]
+
O[15:0]
RTL_ADD
y_reg[16:0]
CLR
C
D
Q
RTL_REG_ASYNC
y[16:0]

Timing

Setup | Hold | Pulse Width

Worst Negative Slack (WNS):	1.611 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	16

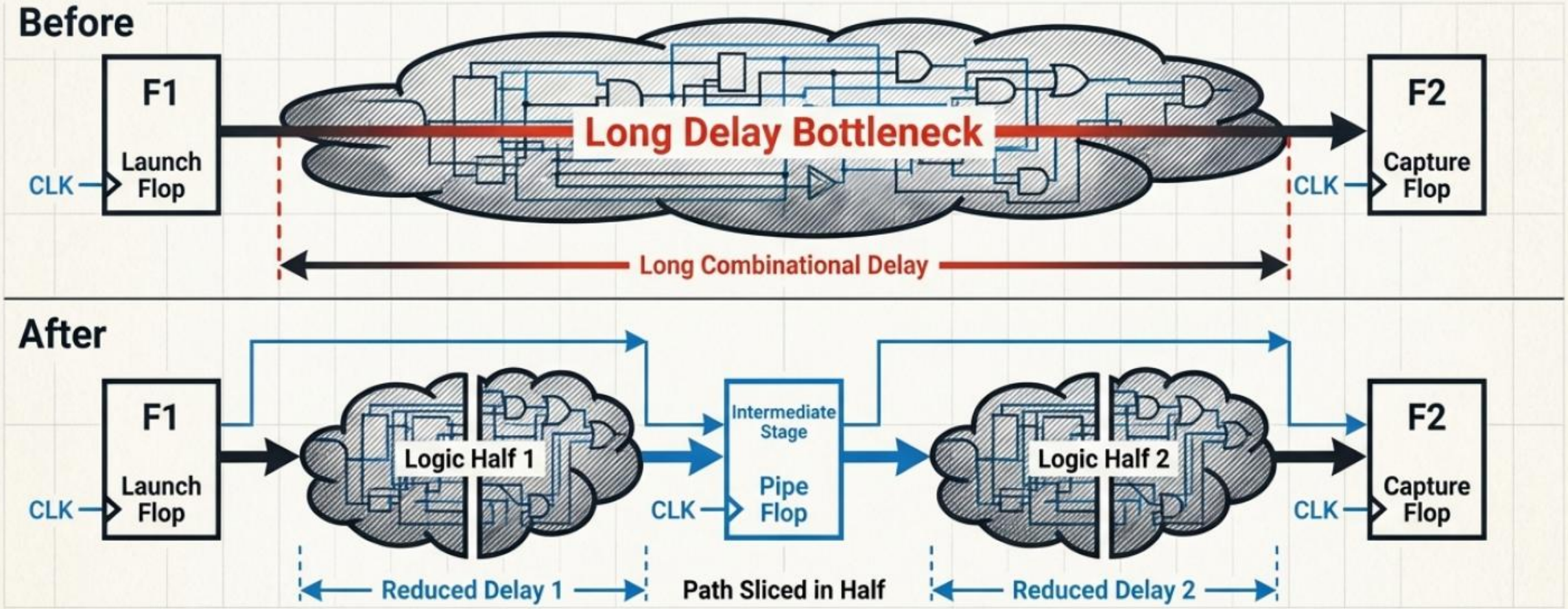
Implemented Timing Report

Timing

Worst Hold Slack (WHS):	0.216 ns
Total Hold Slack (THS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	16

Implemented Timing Report

Architectural Optimization: Pipelining



The Problem

The combinational delay is too long, dictating a severely slow maximum clock frequency.

The Solution

Insert an intermediate flip-flop to cut the logical path in half, creating a multi-stage pipeline.

The Trade-off

Maximum frequency dramatically increases, but it costs physical chip area and adds strict data latency (cycles) to the final output.

verilog

```verilog
always @(posedge clk or posedge rst) begin
    if (rst) begin
        a_reg    <= 0;
        b_reg    <= 0;
        c_reg    <= 0;
        mult_reg <= 0;
        y        <= 0;
    end
    else begin
        a_reg    <= a;
        b_reg    <= b;
        c_reg    <= c;

        mult_reg <= a_reg * b_reg;

        y        <= mult_reg + c_reg;   // no c_delay
    end
end
```

- **Stage 1:** register inputs `a`, `b`, `c`
- **Stage 2:** perform multiplication and register the product; also delay `c`
- **Stage 3:** add registered product with delayed `c` and register output

Latency

This gives **3 clock-cycle latency** from input to final output.

Throughput

After pipeline fills, you can get **one output per clock cycle**.

Datapath

$$y = (a \times b) + c$$

Pipeline form:

$$(a, b, c) \rightarrow \text{Reg} \rightarrow \text{Multiplier} \rightarrow \text{Reg} \rightarrow \text{Adder} \rightarrow \text{Reg}$$

Suppose inputs come as:

- Sample 1: (a_1, b_1, c_1)
- Sample 2: (a_2, b_2, c_2)
- Sample 3: (a_3, b_3, c_3)

Then correct outputs should be:

$$y_1 = a_1 b_1 + c_1$$

$$y_2 = a_2 b_2 + c_2$$

$$y_3 = a_3 b_3 + c_3$$